



AI自动化coding 2-0 [视频][源码]

来自：雷哥AI 解决方案



2026年04月03日 20:53

一、为什么现在是 AI 自动化 Coding 的最佳时机

当前 AI 能力已达到关键拐点，主要体现在两个维度：

- 1. **模型能力显著提升**：近年来大模型在代码理解、生成与调试上的能力大幅增长，已足以承担较复杂的开发子任务。
- 2. **使用成本大幅降低**：过去，性能足够用于编程的模型几乎都是海外商业模型，成本极高。如今国内模型在自动化编程场景下已非常可用，综合成本显著下降。

早期 AI 能力不足时，直接让其完成大量工作极易陷入死循环——频繁报错、无法收敛。而现在，通过合理的流程设计，AI 已可以稳定完成拆解后的子任务。

能力：

AI自动coding一宿：7w+ 行代码，顺利提交pr

提交的PR地址: <https://github.com/DayuanJiang/next-ai-draw-io/pull/765>

DayuanJiang / next-ai-draw-io

Search

Code Issues 113 Pull requests 28 Discussions Actions Projects Models Security and quality Insights

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

feat: replace Quick Examples with personal My Templates library #765

astordu wants to merge 616 commits into DayuanJiang:main from astordu:ralph/my-templates-replaces-quick-examples

Conversation 1 Commits 616 Checks 2 Files changed 265 +71,148 -6,032

astordu commented 4 hours ago

What's Changed

Replaces the static Quick Examples panel with a personal **My Templates** library, allowing users to save, manage and reuse their own prompts.

Features

- **CRUD**: Create, edit, duplicate and delete templates
- **Persistent storage**: IndexedDB-backed, survives page refresh
- **One-click send**: Click a template to fill the chat input
- **Search & sort**: Real-time search; pinned templates stay on top
- **Save from input**: Save current input or any past message as a template
- **Import / Export**: Back up and restore templates as JSON
- **i18n**: Full support for EN / ZH / ZH-Hant / JA

Files Changed

- `lib/template-storage.ts` — new IndexedDB storage layer (with unit tests)
- `components/chat/TemplatePanel.tsx` — main template UI

Reviews

No reviews

Still in progress? [Convert to draft](#)

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

费用：

7天我话费了3亿+token.

使用详情 | 数据非实时，约延迟10分钟

模型用量 工具用量

当天 最近7天 最近30天 2026-03-26 ~ 2026-04-01

TOKENS总量

334,371,155

TOKENS总量

https://articles.zsxq.com/id_xjpq4lwk8zee.html

1/15

而只需要锁定费用5k/年内就可以完成.

连续包月

连续包季 9折

连续包年 8折

连续包年享8折优惠

Lite

连续包年 8 折

¥39.2/月 ¥49/月

下个年度续费金额：¥470.4

特惠订阅

3x Claude Pro 用量额度

✓ 适合小型 Repo 轻量级迭代

✓ 逐步开放最新旗舰模型及功能

✓ 支持 Claude Code 等 20+ 编程工具 ①

Pro

连续包年 8 折

¥119.2/月 ¥149/月

下个年度续费金额：¥1430.4

特惠订阅

5x Lite 用量额度 + Lite 全量权益

✓ 适合中型 Repo 日常开发

✓ 优先体验最新旗舰模型及功能

✓ 覆盖多款精选 MCP 工具

✓ 更快生成速度

Max

连续包年 8 折

¥375.2/月 ¥469/月

下个年度续费金额：¥4502.4

特惠订阅

20x Lite 用量额度 + Pro 全量权益

✓ 适合高阶用户中大型 Repo 深度开发

✓ 首发接入最新旗舰模型及功能

✓ 高峰期专属资源优先保障

二、AI 自动化 Coding 的三大核心问题

全自动化开发涉及三个必须解决的核心问题：

问题	说明
任务拆解粒度	需将任务拆解到 AI 能在单次上下文内独立完成的程度，通过多个小任务的组合实现完整项目
成功标准定义	须为每个任务明确验收标准，自动化程序才能判断何时停止、如何评价质量
成本控制	确保整体任务拆解方案在经济上可行

三、当前 AI 模型的固有限制

在设计解决方案之前，必须清楚认识现有模型的能力边界。

3.1 上下文增大导致性能衰退

研究表明，任何模型在上下文（Context）规模增大时，其性能都会出现显著下降。

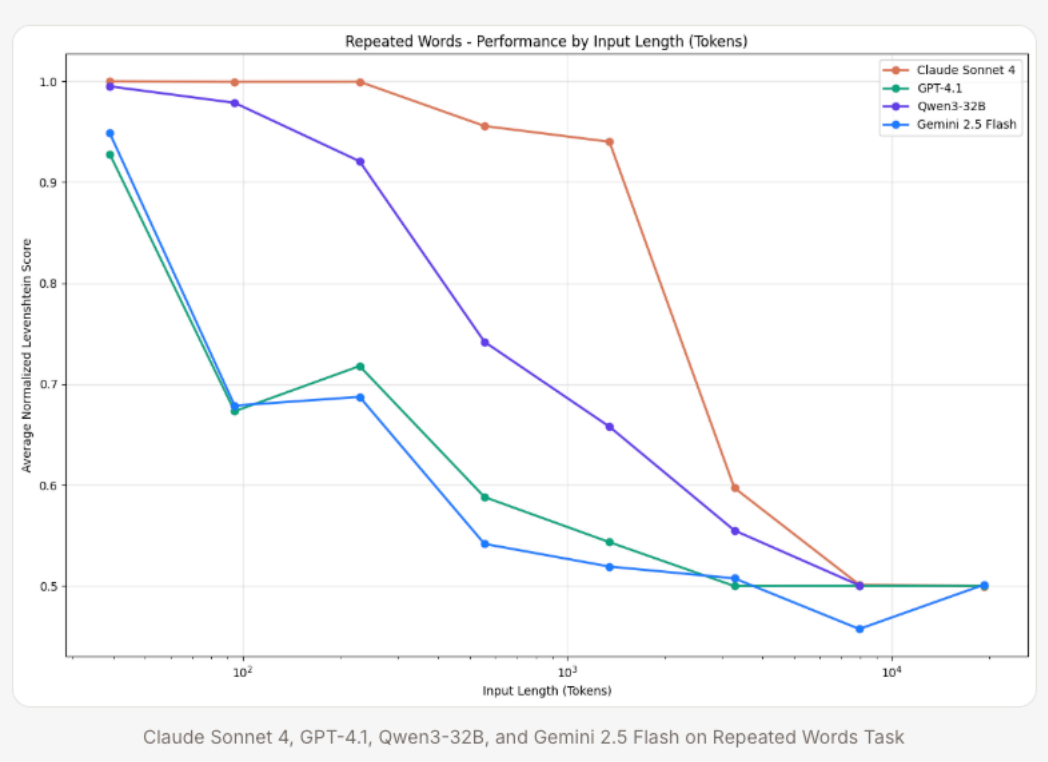
4 Experiments

Models	Claimed	Effective	Base Score (× 0.85: Thr.)	1K	2K	4K	8K	16K	32K
	Length	Length							
GPT-4o	128K	8K	99.3 (84.4)	<u>98.1</u>	<u>98.0</u>	<u>95.7</u>	<u>89.2</u>	81.6	69.7
Llama 3.3 70B	128K	2K	97.3 (82.7)	<u>94.2</u>	<u>87.4</u>	81.5	72.1	59.5	42.7
Llama 3.1 405B	128K	2K	94.7 (80.5)	<u>89.0</u>	<u>85.0</u>	74.5	60.1	48.4	38.0
Llama 3.1 70B	128K	2K	94.5 (80.3)	<u>91.0</u>	<u>81.8</u>	71.2	62.7	51.8	43.2
Gemini 1.5 Pro	2M	2K	92.6 (78.7)	<u>86.4</u>	<u>82.7</u>	75.4	63.9	55.5	48.2
Jamba 1.5 Mini	256K	< 1K	92.4 (78.6)	76.3	74.1	70.8	62.2	52.7	43.6
Command R+	128K	< 1K	90.9 (77.3)	77.0	73.5	66.2	39.5	21.3	7.4
Gemini 2.0 Flash	1M	4K	89.4 (76.0)	<u>87.7</u>	<u>87.5</u>	<u>77.9</u>	64.7	48.2	41.0
Mistral Large 2	128K	2K	87.9 (74.7)	<u>86.1</u>	<u>85.5</u>	73.3	51.4	32.6	18.8
Claude 3.5 Sonnet	200K	4K	87.5 (74.4)	<u>85.4</u>	<u>84.0</u>	<u>77.6</u>	61.7	45.7	29.8
Gemini 1.5 Flash	1M	< 1K	84.7 (72.0)	68.6	61.6	51.0	44.4	35.5	28.6
GPT-4o mini	128K	< 1K	84.8 (72.1)	67.7	58.2	44.2	32.6	20.6	13.7
Llama 3.1 8B	128K	1K	76.7 (65.2)	<u>65.7</u>	54.4	44.1	31.9	22.6	14.2

Table 3: NOLIMA benchmark results on the selected models. Following Hsieh et al. (2024), we report the effective length alongside the claimed supported context length for each model. However, we define the effective length as the maximum length at which the score remains above a threshold, set at 85% of the model’s base score (shown in parentheses). Scores exceeding this threshold are underlined. Scores that are below 50% of the base score are shaded in red.

参考资料：

- 论文：Long-Context Evaluation Beyond Literal Matching — <https://arxiv.org/html/2502.05167v3>
- 文章：Context Rot — <https://www.trychroma.com/research/context-rot>



3.2 上下文焦虑 (Context Anxiety)

部分模型在接近其自身认为的上下文极限时，会提前结束工作，导致任务未完成即收尾。

原文引用 (Anthropic 工程博客)：

"Some models also exhibit 'context anxiety,' in which they begin wrapping up work prematurely as they approach what they believe is their context limit."

来源：<https://www.anthropic.com/engineering/harness-design-long-running-apps>

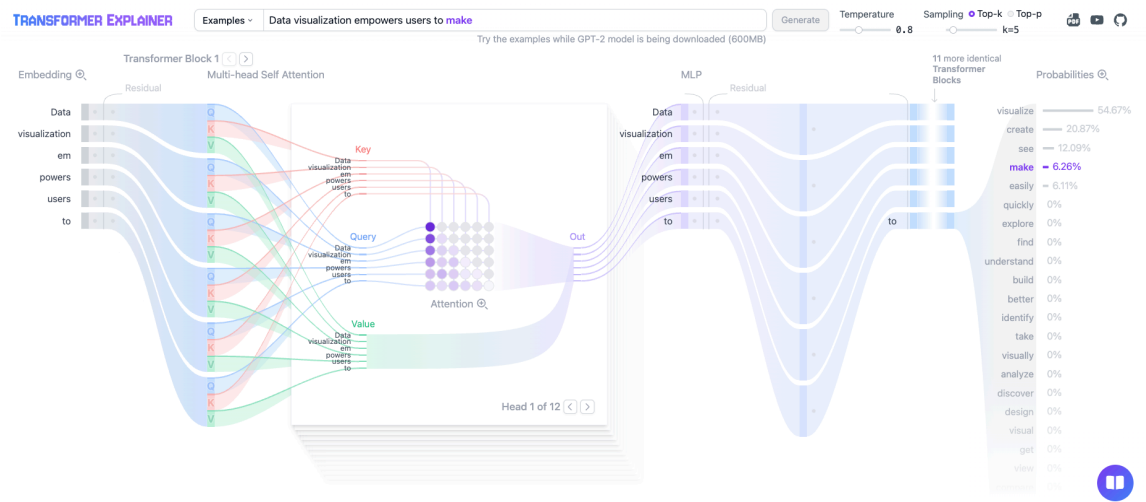
3.3 自我认知局限

模型在同一上下文中，倾向于认同自己已完成的工作，难以从独立视角审视之前的决策。

原文引用（Anthropic 工程博客）：
"When asked to evaluate work they've produced, agents tend to respond by confidently praising the work"
来源：<https://www.anthropic.com/engineering/harness-design-long-running-apps>

3.4 概率性输出问题

模型本质上是概率模型——从 token 层面到句子层面，所有输出均为概率分布的采样结果。这意味着：即使输入完全相同，模型也可能产生不同输出，系统行为难以保证确定性。



Transformer 概率机制可视化工具：<https://poloclub.github.io/transformer-explainer/>

3.5 针对上述限制的解决策略

限制	解决方案
上下文性能衰退	每个任务开启新的对话窗口，避免上下文累积
上下文焦虑	同上，确保每次 Agent 实例只持有当前任务的上下文
自我认知局限	引入独立的 Validator Agent，专门负责验证 Developer Agent 的产出
概率性输出	以固定流程约束 Agent 行为，减少不确定性

关于固定流程的补充说明：这也解释了 AI 落地的最佳实践路径——优先级从低到高依次为：

1. 固定程序/流程优先：能用确定性逻辑实现的，就不要引入 AI
2. 单节点引入 AI：在固定流程的某个环节引入 AI 辅助决策
3. 单一小场景 Agent：若单节点不满足需求，引入 Agent，但 Agent 负责的场景范围要小
4. 技能/上下文扩展：将 Agent 的能力以 Skills 或多上下文的形式扩展
5. 多 Agent 协作（慎用）：多 Agent 是最后手段。它放大了概率问题，同时加剧了上下文衰退、焦虑等问题。当前模型均未从根本上解决这些问题，多智能体目前仍更多停留在理论层面。

四、自动化 Coding 的架构设计

基于上述限制，自动化 Coding 系统至少需要两个角色：

- Developer Agent：负责接收单个任务并执行开发
- Validator Agent：负责独立验证 Developer 的产出

设计要点说明：

- 解决上下文焦虑：Developer 每次只接收一个 User Story，上下文极小，不存在因上下文过长而提前收工的问题。
- 解决自我认知局限：由独立的 Validator Agent 对 Developer 的产出进行审查，避免"自我评判"的偏差。
- 解决概率性问题：开发 → 验证 → 重新开发的循环流程以程序化方式固化，减少 Agent 之间自由指派任务带来的不确定性。

整个流程需通过代码方式固化（程序化编排），确保每次执行路径一致。

五、标准开发流程

5.1 流程概述

自动化开发的核心是**开发流程的自动化**，前提是明确"要做什么"。
无论是新产品还是新功能，都需在开发前将以下问题想清楚：

- 面向什么用户，解决什么问题？
- 功能边界是什么，范围如何界定？
- 验收标准是什么？

从敏捷开发视角看，完整流程为：

PRD → User Story → Sprint → Task

5.2 各层级定义

PRD（产品需求文档）

回答"为什么做、做什么、目标是什么"，包含：

- 背景与目标
- 用户画像
- 问题定义
- 功能范围与业务规则
- 验收标准

User Story（用户故事）

站在用户视角，描述用户行为与价值，格式为：

作为【某类用户】，我想要【做某件事】，以便【获得某种价值】。

示例：

作为电商用户，我想在下单前看到运费，以便决定是否购买。

Sprint（迭代周期）

固定时间盒（通常 1–2 周），团队在此周期内承诺完成一批 User Story。Sprint 是执行周期，而非需求内容本身。

Task（任务）

最细粒度的执行单元，由 User Story 拆分而来，分配给开发、测试、设计等角色执行。

各层级关系一句话总结：

PRD 讲需求全貌，User Story 讲用户价值，Sprint 讲本次迭代做哪些，Task 讲具体谁去做什么。

5.3 结合 AI 构建需求文档

推荐使用能力最强的模型（不限工具，如 Cursor、Claude、Copilot 等）完成以下过程：

1. 让 AI 读取现有项目代码，理解当前架构
2. 在此基础上提出新 Feature 需求

3. 与 AI 进行深入讨论，确保以下核心问题均有明确回答

- 重点关注：
- **问题/目标：** 这解决了什么问题？
 - **核心功能：** 关键操作是什么？
 - **范围/边界：** 它不应该做什么？
 - **成功标准：** 我们如何知道它已完成？

讨论结论将自动形成需求文档，供后续自动化开发使用。

六、最小可执行 User Story

6.1 设计原则

引入 AI 后，不再需要将需求细化到 Task 层级——只需到 **User Story** 层级即可，AI 会自行处理执行细节。
关键在于：**User Story 不能太大**。
单个 Story 的大小须确保 AI 能在一次上下文窗口（一个 Context Window）内独立完成。若 Story 过大，LLM 在完成前会耗尽上下文，产生不完整甚至损坏的代码。

6.2 Story 大小规则

Story 大小：第一规则

每个 story 必须能在一次 Ralph 迭代（一个 context window）中完成。

Ralph 每次迭代都会生成一个新的 Claude code 实例，没有之前工作的记忆。如果 story 太大，LLM 在完成之前会用完 context window。

合适大小的 stories：

- 添加 database 列和 migration
- 向现有页面添加 UI component
- 使用新逻辑更新 server action
- 向列表添加 filter dropdown

太大（需要拆分）：

- "构建整个 dashboard" - 拆分为：schema、queries、UI components、filters
- "添加 authentication" - 拆分为：schema、middleware、login UI、session handling
- "重构 API" - 拆分为每个 endpoint 或 pattern 一个 story

经验法则： 如果你无法用 2-3 句话描述这个变更，那就太大了。

6.3 验收标准

每个 User Story 必须附带明确的验收标准（Acceptance Criteria），用于判断该 Story 是否成功完成。验收标准可以是：

- ✓ Linter/TypeCheck 检查通过
- ✓ 数据库单元测试通过
- ✓ 浏览器特定交互行为验证通过

验收标准示例：

```
{
  "id": "US-009",
  "title": "数据重置按钮",
```

```
"description": "作为用户，我可以通过专门的按钮一键将数据恢复为初始示例状态，方便演示。",
"acceptanceCriteria": [
  "页面底部角落放置\"重置到示例数据\"按钮（配置 aria-label）",
  "点击后无刷新将内存状态恢复为初始 2-3 条示例笔记",
  "重置后默认选中第一条示例笔记",
  "搜索框同时清空",
  "Typecheck passes",
  "Verify in browser using agent-browser"
],
"priority": 9,
"passes": true,
"notes": "",
"retryCount": 0,
"blocked": false
}
```

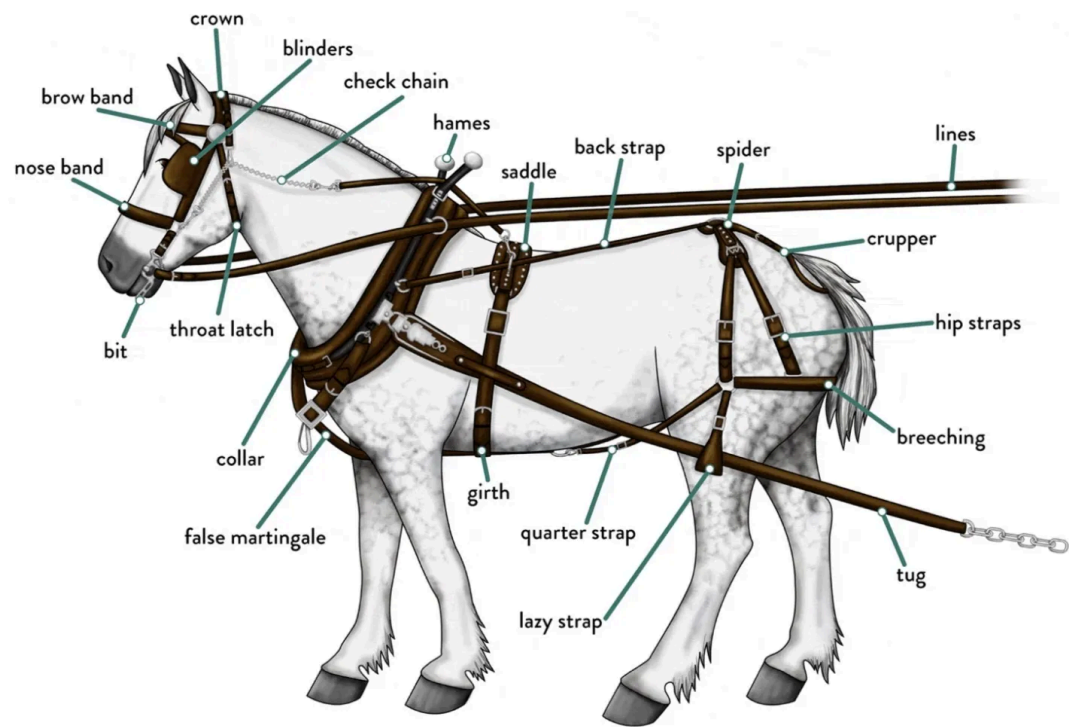
七、Harness 工程：约束 AI 行为的核心机制

7.1 什么是 Harness

AI 每次执行 User Story 时，需要读取必要的上下文，才能准确完成任务、减少错误。这套提供给 AI 的上下文约束集合，称为 **Harness 工程**。



图片来源: <https://a-z-animals.com/animals/horse/>



图片来源: <https://cornerstoneranch.org/blog/harness-facts/>
标准 Harness 文件的代表是 CLAUDE.md, 用于帮助 AI 理解项目结构与开发规范。在此基础上, 可以扩展引用其他规范文件, 形成渐进式加载的约束体系。
Harness 的目标: 减少 AI 在自动化开发过程中的错误, 提供明确的方向, 使开发更加准确。

7.2 Harness 的两种来源

来源一: 预先设计的规范
将已知的规范直接写入约束文件, 例如:

- 头文件命名规则
- 数据库表设计规范
- 代码风格要求

来源二: 从错误中沉淀的规则
在与 AI 协作开发的过程中, 发现代码不符合预期后, 要求 AI 修正, 并将修正后的规则沉淀为 Harness 条目。这类规则来源于真实协作中的问题, 往往更具针对性。
常见的需要沉淀的问题类型:

- AI 修改了某个功能, 但忘记更新联动状态
- AI 通过了 TypeCheck, 但浏览器点击无反应
- AI 未遵守代码库已有的模式
- AI 为完成局部目标, 引入了全局回归
- AI 在环境、端口、测试依赖上判断失误

关键洞察: 如果这些问题只是"当场修掉", 经验很快就会丢失。只有将其沉淀为规则、模式、验证脚本, 它们才会成为未来自动化的燃料。

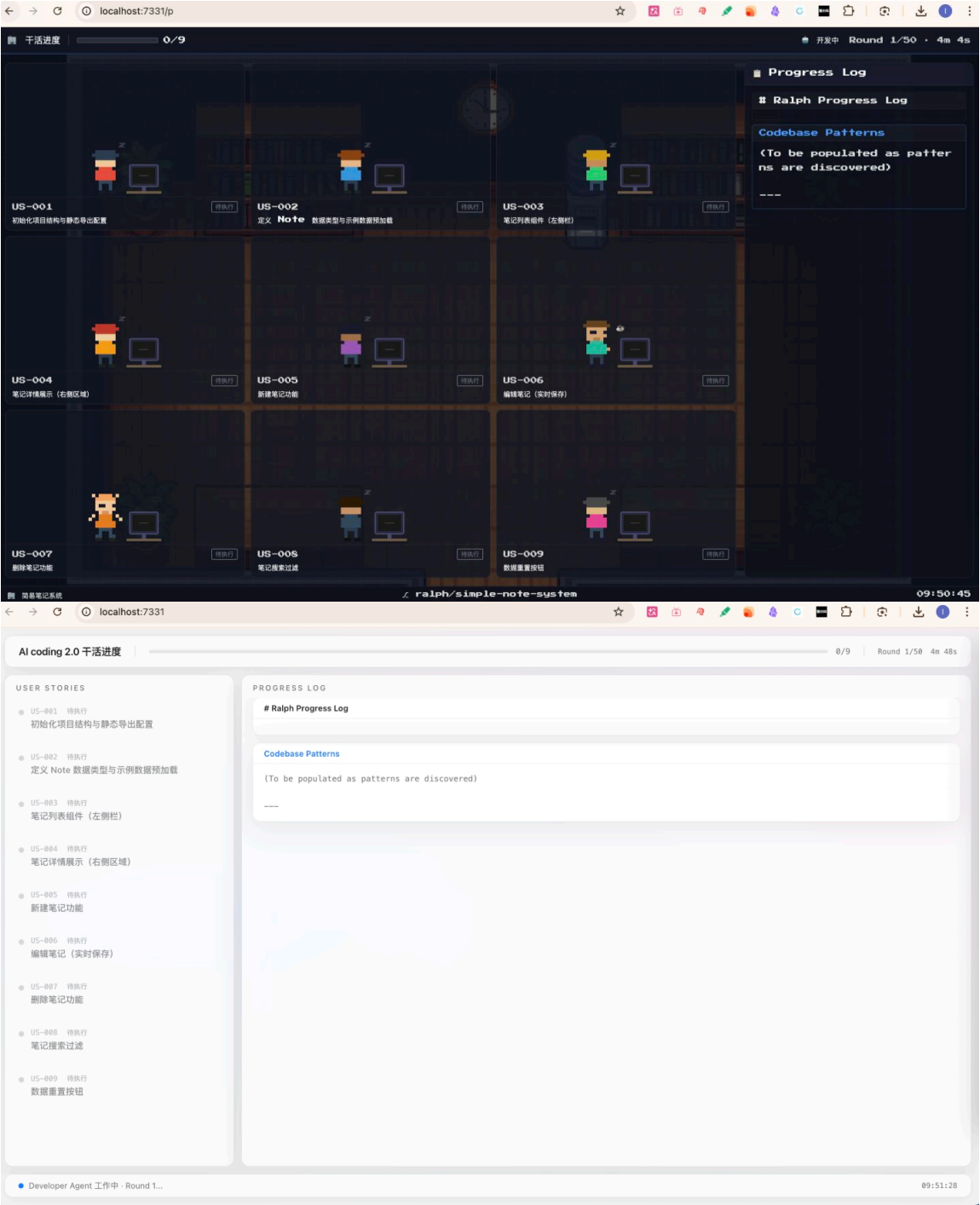
7.3 Harness 的迭代策略

两类规范需要持续积累与迭代。随着 Harness 不断完善, AI 的执行效率和准确性会逐步提升。
建议: 若当前 Harness 积累尚不充分, 建议从 Copilot 模式 (人机协作) 起步, 而非直接追求完全自治。先在协作中积累经验, 再逐步向自动化迁移。

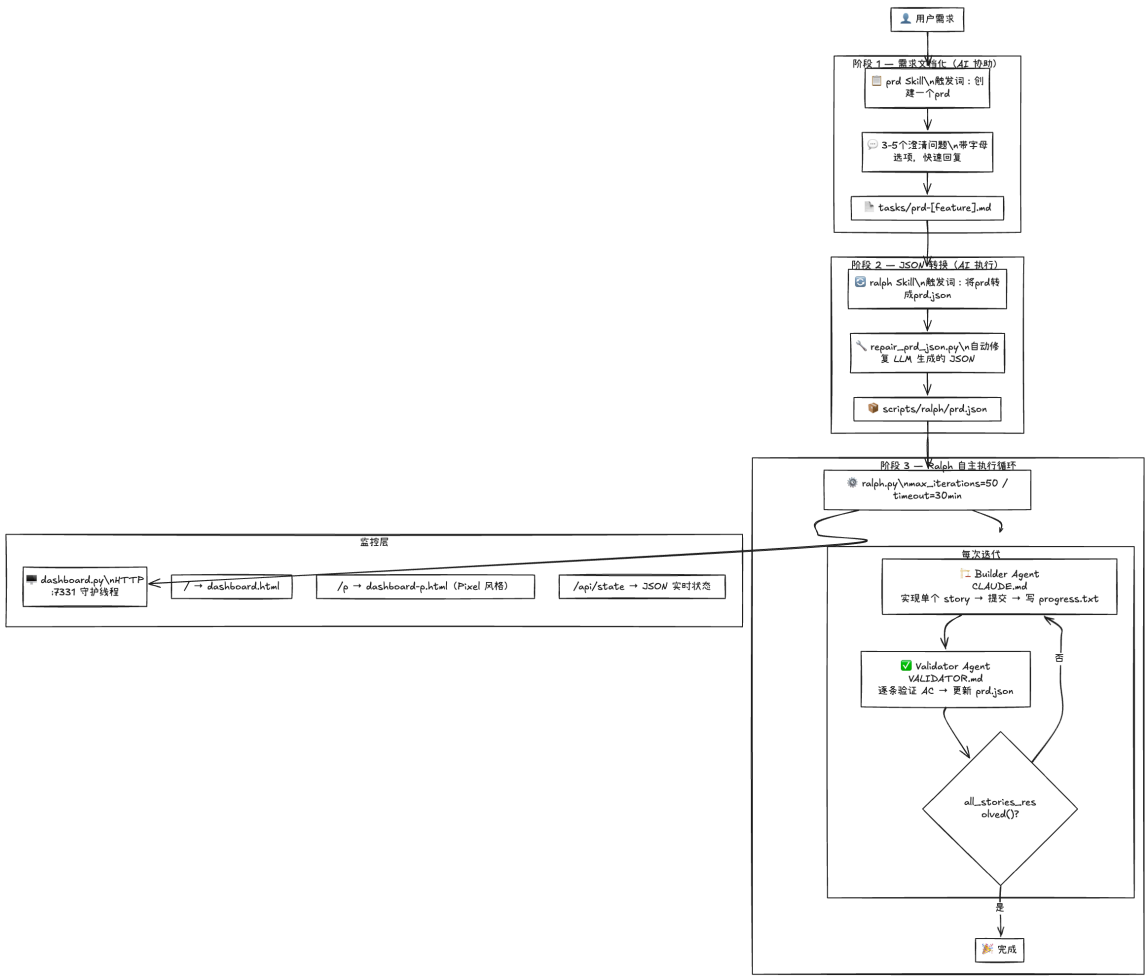
八、Auto Coding 系统完整架构分析

8.0 增加了监控系统

启动ralph.py时候, 系统会自动打开网页: http://localhost:7331
默认是简洁的页面, 如果想看到像素人办公, 可以在地址后边加入/p: http://localhost:7331/p



8.1 系统全景概览



8.2 六个层次详解

层 1 | prd Skill — 需求文档化

文件: .agents/skills/prd/SKILL.md

步骤	内容
接收需求	用户描述功能
澄清问题	3-5 个带字母选项的问题, 用户可快速回复 1A, 2C
生成 PRD	结构化 Markdown (含 US-001 格式 + 验收标准)
保存	tasks/prd-[feature-name].md

关键规则:

- AC 必须可验证 (「点击删除显示确认对话框」✅ vs 「工作正常」❌)
- 有 UI 变更的 story 必须包含「使用 agent-browser 在浏览器中验证」
- 每条 story 末尾必须有 Typecheck passes

层 2 | ralph Skill + repair 脚本 — PRD 转换

文件: .agents/skills/ralph/SKILL.md + .agents/skills/ralph/scripts/repair_prd_json.py

prd.json 核心字段说明:

字段	含义	谁写

priority	执行顺序（数字小先执行）	ralph Skill
passes	Builder 完成→true; Validator 失败→false	Builder / Validator
notes	Validator 写失败详情, Builder 下次据此修复	Validator
retryCount	每次失败 +1, ≥5 触发 blocked	Validator
blocked	true = 跳过该 story	Validator

JSON 修复流程：

```
AI 生成 prd.json
→ json_repair.repair_json() 修复未转义引号/多余逗号/括号不匹配
→ json.loads() 二次验证
→ 修复失败则报错退出（不覆盖原文件）
```

归档机制：若旧 prd.json 的 branchName 与新功能不同，先归档至 archive/YYYY-MM-DD-feature/，再写入新文件。

层 3 | ralph.py — 主执行引擎

文件：scripts/ralph/ralph.py

主循环逻辑：

```
ralph.py 启动
↓
dashboard.start()      ← 守护线程，端口 7331，自动打开浏览器
↓
for i in 1..50:
    get_current_story_id() ← 最高 priority 且 passes=false/blocked=false
    dashboard.set_state(phase="developing")
    run_developer(i)      ← 调用 Builder (CLAUDE.md)，超时 30min
        └─ 超时 → set_state("idle") → continue (跳过 Validator)
        └─ 正常完成 ↓
    dashboard.set_state(phase="validating")
    run_validator(i)      ← 调用 Validator (VALIDATOR.md)，超时 60min
    dashboard.set_state(phase="idle")
    all_stories_resolved()
        └─ 否 → 下一轮
        └─ 是 → set_state("done") → sys.exit(0)
```

PTY 机制：用 script -q /dev/null 包装命令，确保 Claude/Codex 的输出实时流式显示在终端。

层 3a | Builder Agent (CLAUDE.md)

文件：scripts/ralph/CLAUDE.md

```
1. 读 prd.json → 找最高 priority 的未完成 story
2. 若 notes ≠ "" → 优先按失败原因修复（不重新实现）
3. 读 progress.txt 顶部 Codebase Patterns（避免重复错误）
4. 切换/创建 Git 分支 (branchName)
5. 只实现这一个 story
6. 运行 typecheck/lint/test
```

```
7. git commit: feat: [US-001] - [Title]
8. prd.json 中 passes = true
9. progress.txt 追加（永不替换）
```

progress.txt 双区结构：

```
## Codebase Patterns      ← 顶部，跨迭代复用
- migrations 始终用 IF NOT EXISTS
- 从 actions.ts 导出 types 供 UI 使用

## 2024-04-01 14:30 - US-001
- 实现了...
- 更改了...
- **未来迭代的学习：**
  - patterns / 陷阱 / 上下文
---
```

层 3b | Validator Agent (VALIDATOR.md)

文件：scripts/ralph/VALIDATOR.md

```
1. 读 progress.txt → 最后一个 ## section → 提取 Story ID
2. 读 prd.json → 找该 story 的 acceptanceCriteria
3. 逐条验证：
  - "Typecheck passes"      → npm run typecheck
  - "Verify in browser"    → agent-browser 实际操作
  - 其他                    → 代码检查 + 浏览器
4. 更新 prd.json
5. 截图保存 screenshots/validator-[us-id]-[pass/fail]-[n].png
```

写回规则：

情况	passes	notes	retryCount	blocked
全部通过	保持 true	清空	重置 0	不变
任意失败	→ false	写失败详情	+1	不变
retryCount ≥ 5	→ false	追加 BLOCKED	+1	→ true

层 4 | Dashboard 监控

文件：scripts/ralph/dashboard.py + dashboard.html + dashboard-p.html

路由	内容
GET /	dashboard.html（标准监控视图）
GET /p	dashboard-p.html（Pixel 风格，312KB）
GET /api/state	JSON 实时状态

/api/state 响应结构：

```
{
  "runtime": {
    "iteration": 3,
    "max_iterations": 50,
    "phase": "validating",
    "current_story": "US-002",
    "elapsed": 1234
  },
  "project": "任务应用",
  "branchName": "ralph/task-status",
  "stories": [ /* prd.json 的 userStories */ ],
  "logs": "progress.txt 全文"
}
```

状态机: idle → developing → validating → idle → ... → done

层 5 | agent-browser 浏览器验证

文件: .agents/skills/agent-browser-skill/SKILL.md
Builder 和 Validator 均可调用, 核心用法:

```
agent-browser open <url>
agent-browser snapshot -i           # 返回 @e1, @e2 等可交互元素 refs
agent-browser click @e1
agent-browser fill @e2 "text"
agent-browser screenshot body out.png
agent-browser record start ./demo.webm
```

避免的反模式 (在 CLAUDE.md 和 VALIDATOR.md 中均有约束):

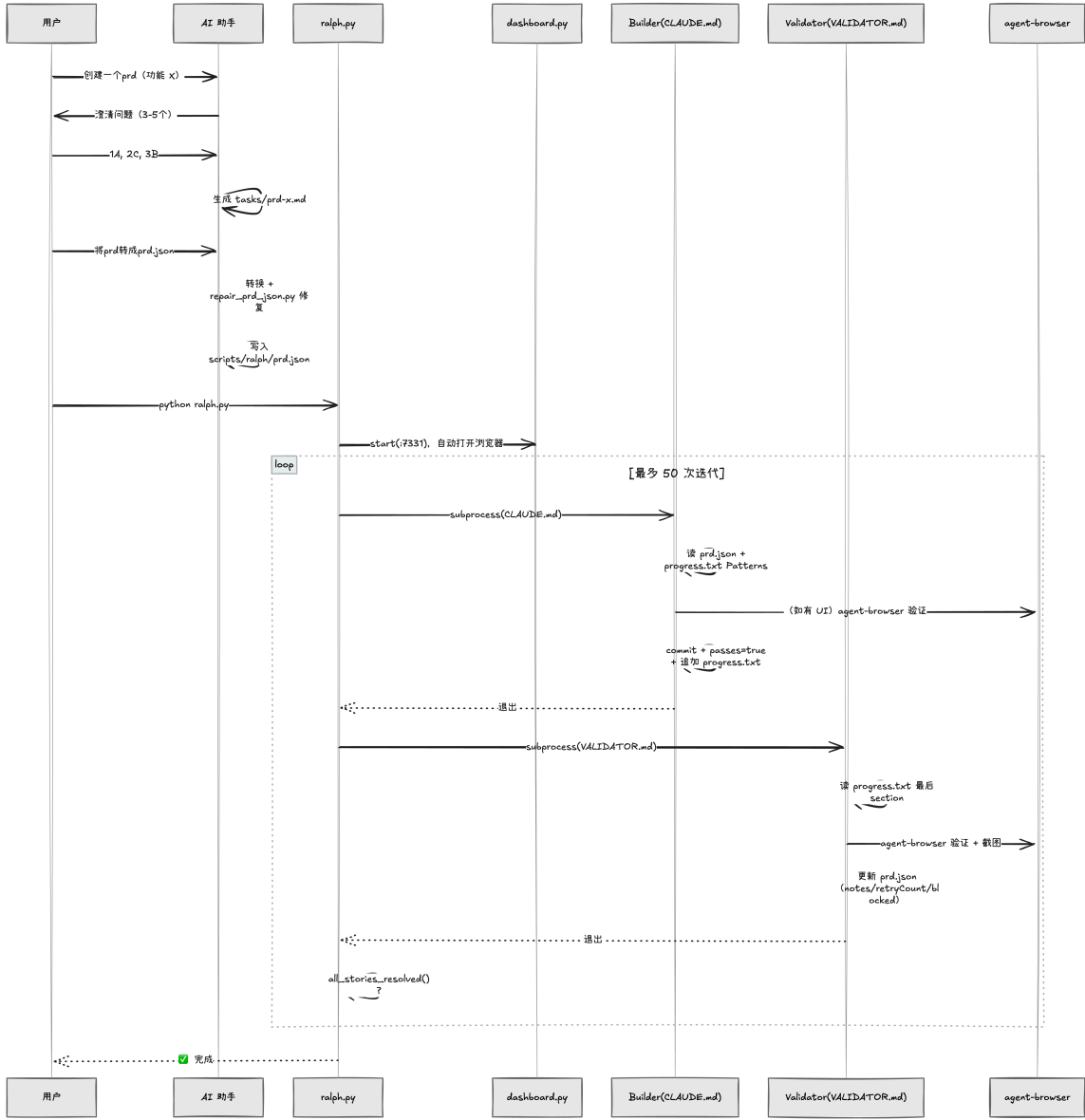
- ✗ 每次迭代重启 dev server
- ✗ 默认使用 kill -9
- ✓ 先检查端口是否可访问, 可访问直接复用

层 6 | Commands 命令层

命令文件	触发	功能
prime.md	/prime	初始化代码库理解 (git ls-files + 读文档)
plan-feature.md	/plan-feature <功能>	5 阶段深度功能规划 → .agents/plans/xx.md
create-rules.md	/create-rules	分析代码库 → 生成 AGENTS.md 全局规则

/plan-feature 的 5 阶段: 理解功能 → 代码库情报 → 外部调研 → 策略思考 → 生成计划

8.3 完整端到端 Sequence



8.4 1.0版本 vs 2.0版本

内容	1.0版本	2.0版本
需求 → PRD 文档	✅	✅ 含澄清问题机制
prd Skill	❌ 缺失	✅ 完整描述
PRD → prd.json 转换	❌ 缺失	✅ ralph Skill + repair 脚本
ralph.py 内部结构	⚠️ 笼统	✅ Builder + Validator 双 Agent
Dashboard 监控	❌ 缺失	✅ dashboard.py + 两个 HTML 视图 + API
agent-browser 验证层	❌ 缺失	✅ 完整约束规则
重试/容错机制	❌ 缺失	✅ retryCount/blocked/超时
Commands 层	❌ 缺失	✅ prime/plan-feature/create-rules

8.5 总结

这套系统的核心设计哲学是 「上下文为王 + 迭代验证」：

- 1. **prd.json** 是所有状态的单一真相源（single source of truth）——Builder 读它知道做什么，Validator 写它记录结果
- 2. **progress.txt** 是跨迭代的记忆载体——Codebase Patterns 区避免重复踩坑，迭代日志区供 Validator 定位最新工作
- 3. **notes 字段** 是 Validator → Builder 的精准反馈通道——不重新实现，只针对性修复
- 4. **Dashboard** 是人类的观察窗口——通过 /api/state 轮询，实时呈现进度、stories 状态、日志

九、实战案例

9.1 0-1项目

程序员面试助理：

项目	一句话描述	亮点
AI 简历生成器	输入工作经历，AI 帮你生成多种风格简历并可导出 PDF	实用、视觉好看、PDF导出有技术含量

补充功能：

AI 面试题生成器	输入 JD，生成面试题 + 参考答案	程序员刚需，传播快
-----------	--------------------	-----------

而且这些数据要记录我的sqlite数据库中。

重要流程：

- 1. 需求整理
- 2. 页面制作(lovable <https://lovable.dev/> 或者 fronted-skills <https://github.com/anthropics/skills/blob/main/skills/frontend-design/SKILL.md>)
- 3. 根据需求和前端项目生成PRD (审核很重要)
- 4. PRD拆分小story
- 5. 自动化完成story和自动化验证 (检查本次claude和codex状态, 需要和我们自己使用的状态一直)
- 6. 项目完成

9.2 老项目

见视频讲解。

源码视频讲解

通过网盘分享的文件： AI 自动化coding 2.0

链接: <https://pan.baidu.com/s/1H-gR38ZpSt99RUsmCbKLig> 提取码: pdwu

--来自百度网盘超级会员v5的分享